

DEPLOIEMENT D'AWX-OPERATOR AVEC CLUSTER MINIKUBE

INTRODUCTION

De nos jours, nous voyons bel et bien comment les différents équipements sont interconnectés au sein d'une infrastructure informatique. Le suivi, la maintenance de ces derniers passent par l'utilisation de bons outils. Il existe plusieurs outils qui permettent d'assurer cette tâche critique dont notamment : GLPI, awx, etc... tout au long de ce tutoriel nous allons utiliser l'outil AWX-OPERATOR.

1. DIMENSIONNEMENT DU SERVEUR

Afin d'installer AWX-OPERATOR correctement, il est essentiel que le serveur soit doté des caractéristiques ci-après :

ID	RESSOURCES	DESCRIPTION
1	RAM	Pour la charge de travail au niveau du serveur
2	CPU	Processeur
3	DISQUE	Pour le stockage de données
4	RESEAU	Pour la mise en réseau
5	ARCHITECTURE	Pour organiser, sécuriser et optimiser le fonctionnement d'un serveur

Tableau 1 : dimensionnement du serveur

2. COMMENT DECOUVRIR LES INFORMATIONS RELATIVES A SON SERVEUR ?

Pour retrouver les informations relatives au serveur il faut saisir les commandes ci-après :

ID	COMMANDES	DESCRIPTION
1	uname -a	Pour afficher les informations complètes du système Linux.

2	free -h	Commande qui sert à afficher l'état de la mémoire RAM et du swap du système Linux.
3	lscpu	Cette commande sert à afficher toutes les informations détaillées sur le processeur d'un système Linux.

Tableau 2 : information relative au serveur (disque, taille mémoire, architecture, etc...)

Nota : awx-operator sera déployé sur un système d'exploitation linux.

Objectif du dimensionnement : permettre l'exécution et l'utilisation d'awx-operator sans crash. Awx-operator est un outil open source qui consomme énormément des ressources. D'où le bon dimensionnement du serveur afin de permettre l'exécution de ce dernier sans problème.

3. DESCRIPTION DE L'OUTIL AWX-OPERATOR

AWX-OPERATOR est un outil open source développé et maintenu par Red Hat qui permet de manager une infrastructure de façon plus structurée, planifiée grâce à sa capacité d'automatisation, d'intégration de différentes solutions, de synchronisation. C'est un outil qui rend simple l'utilisation d'ansible mais en mode graphique.

4. MODE DE FONCTIONNEMENT ET CHOIX DU CLUSTER

Awx-operator fonctionne sur un cluster kubernetes ou openshift.

Parmi le cluster de kubernetes il existe kubeadm, kind, minikube, ELK, EKS, AKS, etc...

Dans le cadre de ce tutoriel nous allons utiliser le cluster MINIKUBE.

Pourquoi minikube ? Etant dans un environnement de test, l'utilisation d'awx-operator se fera grâce à un cluster minikube. Ce dernier permet de faire une installation en local, de casser, de réparer sans problème car le but est de maîtriser le fonctionnement d'awx-operator qui est un outil qui ne diffère pas de celui mis en œuvre par la firme principale pour entreprise AAP. (Outil payant)



Image 1 : Image du cluster minikube.

5. INSTALLATION DU CLUSTER MINIKUBE

Avant d'installer minikube il faut respecter les prérequis ci-dessous :

1. Bien dimensionner son serveur afin que ce dernier ne crache pas
2. Installer docker engine si non présent. (docker-compose facultatif)

5.1 Dimensionnement du serveur

Afin d'exécuter awx correctement, il faut avoir les prérequis nécessaires

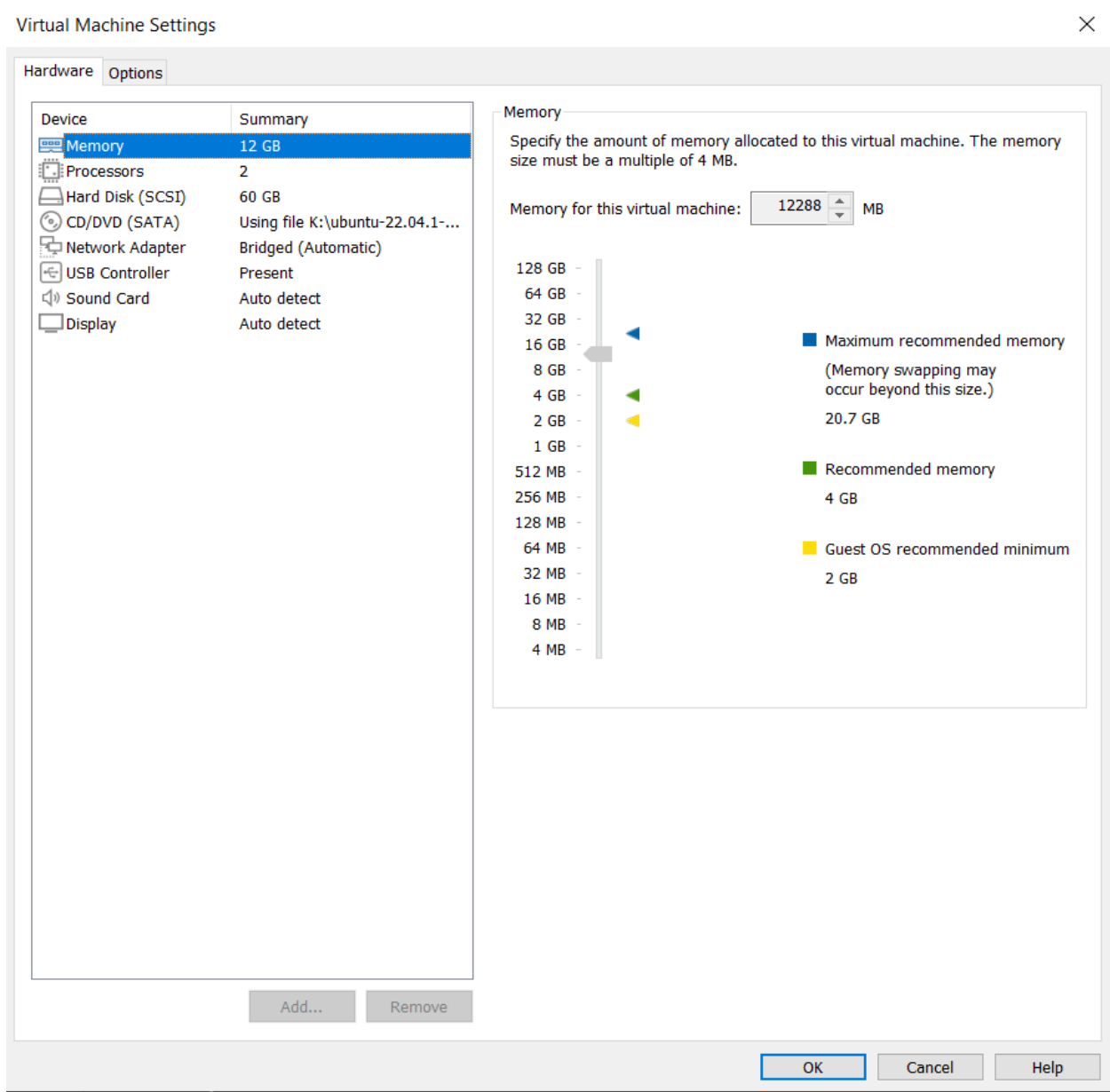


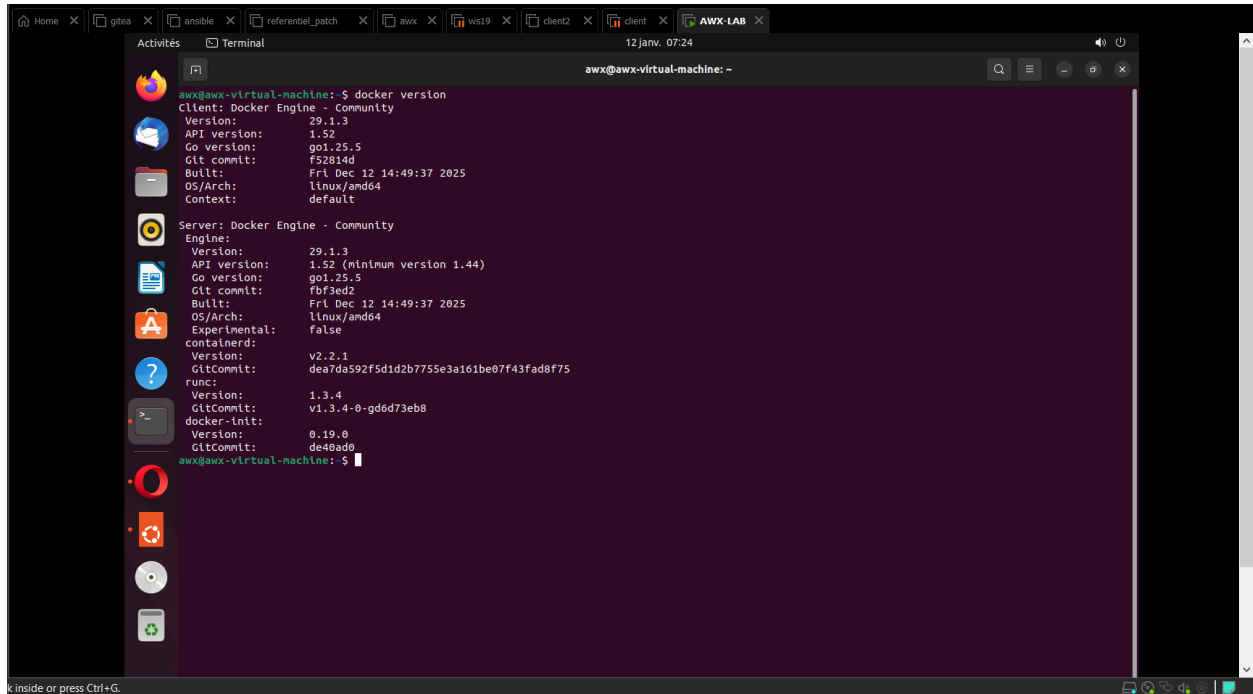
Image 2 : dimensionnement du serveur awx

Nota : Compte tenu de la charge du travail, il est mieux que le serveur dispose d'assez des ressources. Cela permet le bon fonctionnement de l'outil et évite tout dysfonctionnement lorsque la charge du travail s'élève petit à petit.

5.2 Vérification et Installation de docker

Comment s'assurer que docker est déjà installé ?

En saisissant la commande au niveau du terminal : docker version



```
awx@awx-virtual-machine: ~  
12 Janv. 07:24  
awx@awx-virtual-machine: ~  
$ docker version  
Client: Docker Engine - Community  
Version: 29.1.3  
API version: 1.52  
Go version: go1.25.5  
Git commit: f52814d  
Built: Fri Dec 12 14:49:37 2025  
OS/Arch: linux/amd64  
Context: default  
  
Server: Docker Engine - Community  
Engine:  
Version: 29.1.3  
API version: 1.52 (minimum version 1.44)  
Go version: go1.25.5  
Git commit: fbf3ed2  
Built: Fri Dec 12 14:49:37 2025  
OS/Arch: linux/amd64  
Experimental: false  
containerd:  
Version: v2.2.1  
GitCommit: dea7da592f5d1d2b7755e3a161be07f43fad8f75  
runc:  
Version: 1.3.4  
GitCommit: v1.3.4-0-gd6d73eb8  
docker-init:  
Version: 0.19.0  
GitCommit: de40ade  
awx@awx-virtual-machine: ~
```

Image 3 : Vérification de docker installé

Nota : si après l'exécution de cette commande, rien ne s'affiche veuillez passer à l'installation de docker au niveau de votre serveur. Assurez-vous d'avoir une connexion internet stable.

5.3 Installation de docker engine

Pour installer docker, veuillez suivre la procédure via le lien officiel ci-après :

<https://docs.docker.com/engine/install/ubuntu/>

Nous avons installé docker en utilisant le repository apt.

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Image 4 : installation de docker avec le repository apt

Pour plus d'info sur l'installation de docker, veuillez consulter le lien ci-après :

<https://docs.docker.com/engine/install/ubuntu/>

Une fois docker installé, on peut maintenant procéder à la création du cluster MINIKUBE.

Comme mentionné ci-haut, avant d'installer minikube comme cluster il faut connaître les spécificités techniques du serveur. (RAM, CPU, TAILLE DISQUE, TAILLE DE LA MEMOIRE, LINUX COMME SYSTEME D'EXPLOITATION, etc...)

Nota : assurez-vous d'avoir une connexion internet stable afin de créer le cluster minikube sans problème.

6. INSTALLATION DE MINIKUBE

Pour installer minikube comme cluster il faut connaître les éléments ci-après au niveau du serveur

ID	ELEMENTS	COMMENTAIRES
1.	Quel système d'exploitation ?	Linux ? windows ? mac ?
2.	Quelle est l'architecture du serveur ?	Vérifier avec la commande : Uname -a ou lscpu au niveau du terminal
3.	Quel release ?	Est-ce une version stable ?
4.	Quel type d'installation ?	Est-ce une installation binaire ? par paquet ? (debian ou RPM ?)

Tableau numéro 3 : éléments nécessaires pour installer minikube

Nota : l'installation du cluster minikube ne peut se faire qu'en dégageant les différents axes relatifs du tableau ci-haut.

Via cette image, on peut voir bien voir l'architecture, le nombre de cpu du serveur avant l'installation de minikube.

```

awx@awx-virtual-machine:~$ lscpu
Architecture: x86_64
Mode(s) opératoire(s) des processeurs: 32-bit, 64-bit
Address sizes: 45 bits physical, 48 bits virtual
Boutisme: Little Endian
Processeur(s): 2
Liste de processeur(s) en ligne: 0,1
Identifiant constructeur: GenuineIntel
Nom de modèle: 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
Famille de processeur: 6
Modèle: 140
Thread(s) par cœur: 1
Cœur(s) par socket: 1
Socket(s): 2
Révision: 2
BogoMIPS: 5099.40
Drapeaux: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss syscall nx pdpe
1gb rdtscp lm constant tsc arch perfmon rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pc
lmulqdg ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor
r lahf_lm abm 3dnowprefetch invpcid_single ssbd ibrs ibpb stibp ibrs_enhanced fsgsbase tsc_adjust bmi1 avx2 smep bmi
2 erms invpcid avx512f avx512dq rdseed adx smap avx512fma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveop
t xsaves xgetbv1 xsaves orat avx512vbmi umip pku ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg a
vx512_vpopcntdq rdpid movdiri movdir64b fsrm avx512_vp2intersect md_clear flush_l3d arch_capabilities

Virtualization features:
Constructeur d'hyperviseur: VMware
Type de virtualisation: complet
Caches (sum of all):
L1d: 96 KiB (2 instances)
L1i: 64 KiB (2 instances)
L2: 2,5 MiB (2 instances)
L3: 12 MiB (2 instances)
NUMA:
Nœud(s) NUMA: 1
Nœud NUMA 0 de processeur(s): 0,1
Vulnérabilités:
Itlb multihit: KVM: Mitigation: VMX unsupported
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl and seccomp

```

Image 5 : architecture et nombre cpu via la commande lscpu

Nota : hormis cela, il faut également connaître la mémoire disponible. Cela est possible avec la commande : `free -h`

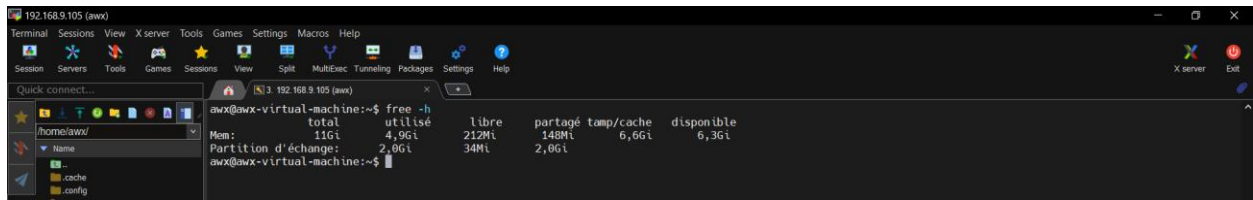


Image 6 : affichage de la mémoire disponible au niveau du serveur

Une fois fait, veuillez procéder à l'installation du cluster minikube proprement.

Via cette image, on voit que :

L'étape 1 : constitue à déterminer l'architecture, le type du système d'exploitation, le type de la release ainsi que le mode d'installation.

L'étape 2 : constitue à télécharger et installer minikube

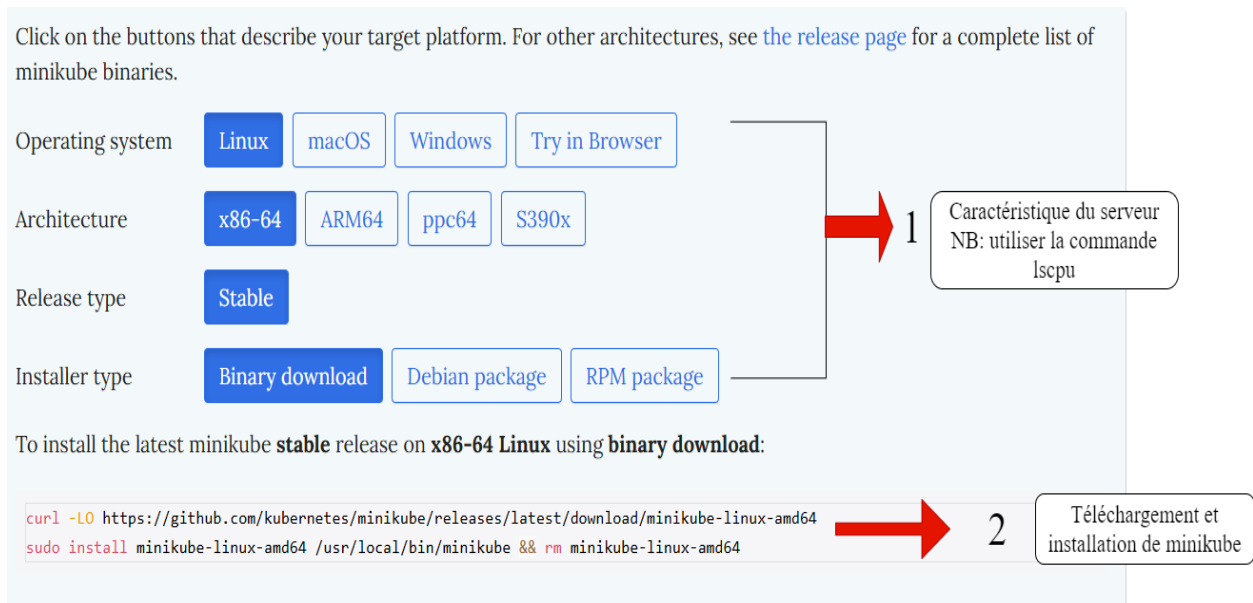
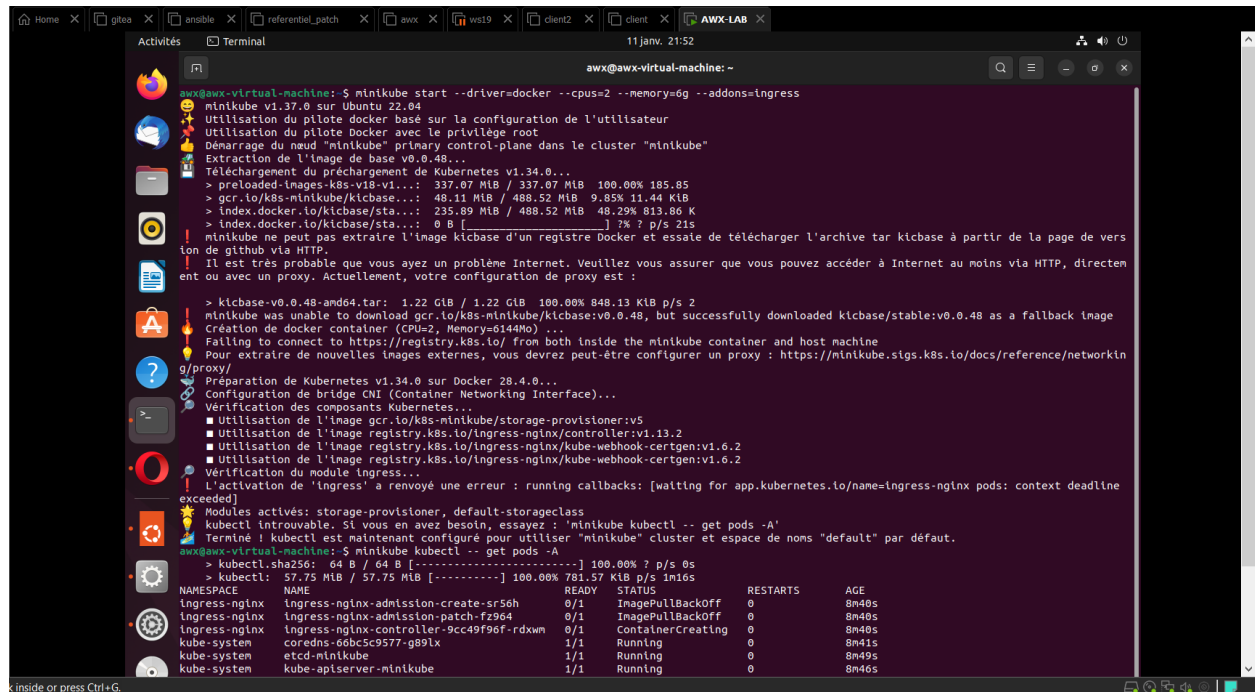


Image 7 : Etape pour le téléchargement de minikube

Une fois l'étape 1 et 2 terminé, démarré le cluster minikube en précisant les options supplémentaires (driver docker, etc...) pour une bonne stabilité. Voir l'image ci-dessous :



```
awx@awx-virtual-machine: ~  
minikube v1.37.0 sur Ubuntu 22.04  
Utilisation du pilote docker basé sur la configuration de l'utilisateur  
Utilisation du pilote Docker avec le privilège root  
Démarrage du nœud "minikube" primary control-plane dans le cluster "minikube"  
Extraction de l'image de base v0.0.48...  
Téléchargement du préchargement de Kubernetes v1.34.0...  
> preloaded-images-k8s-v18-v1...: 337.07 MiB / 337.07 MiB 100.00% 185.85  
> gcr.io/k8s-minikube/kicbase...: 48.11 MiB / 488.52 MiB 9.85% 11.44 KiB  
> index.docker.io/kicbase/sta...: 235.89 MiB / 488.52 MiB 48.29% 813.86 K  
> index.docker.io/kicbase/sta...: 0 B [ ] 7% ? p/s 21s  
minikube ne peut pas extraire l'image kicbase d'un registre Docker et essaie de télécharger l'archive tar kicbase à partir de la page de vers  
ion de github via HTTP.  
Il est très probable que vous ayez un problème Internet. Veuillez vous assurer que vous pouvez accéder à Internet au moins via HTTP, directem  
ent ou avec un proxy. Actuellement, votre configuration de proxy est :  
  
> kicbase-v0.0.48-amd64.tar: 1.22 GiB / 1.22 GiB 100.00% 848.13 KiB p/s 2  
minikube was unable to download gcr.io/k8s-minikube/kicbase:v0.0.48, but successfully downloaded kicbase/stable:v0.0.48 as a fallback image  
Création de docker container (CPU=2, Memory=6144Mo) ...  
Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine  
Pour extraire de nouvelles images externes, vous devrez peut-être configurer un proxy : https://minikube.sigs.k8s.io/docs/reference/networkin  
g/proxy/  
Préparation de Kubernetes v1.34.0 sur Docker 28.4.0...  
Configuration de bridge CNI (Container Networking Interface)...  
Vérification des composants Kubernetes...  
■ Utilisation de l'image gcr.io/k8s-minikube/storage-provisioner:v5  
■ Utilisation de l'image registry.k8s.io/ingress-nginx/controller:v1.13.2  
■ Utilisation de l'image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2  
■ Utilisation de l'image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2  
Vérification du module ingress...  
! L'activation de 'ingress' a renvoyé une erreur : running callbacks: [waiting for app.kubernetes.io/name=ingress-nginx pods: context deadline  
exceeded]  
Modules activés: storage-provisioner, default-storageclass  
kubectl introuvable. Si vous en avez besoin, essayez : 'minikube kubectl -- get pods -A'  
Terminé ! kubectl est maintenant configuré pour utiliser "minikube" cluster et espace de noms "default" par défaut.  
awx@awx-virtual-machine: ~$ minikube kubectl -- get pods -A  
> kubectl sha256: 64 B / 64 B [-----] 100.00% ? p/s 0s  
> kubectl: 57.75 MiB / 57.75 MiB [-----] 100.00% 781.57 KiB p/s 1m16s  


| NAMESPACE     | NAME                                     | READY | STATUS            | RESTARTS | AGE   |
|---------------|------------------------------------------|-------|-------------------|----------|-------|
| ingress-nginx | ingress-nginx-admission-create-sr56h     | 0/1   | ImagePullBackOff  | 0        | 8m40s |
| ingress-nginx | ingress-nginx-admission-patch-fz964      | 0/1   | ImagePullBackOff  | 0        | 8m40s |
| ingress-nginx | ingress-nginx-controller-9cc49f96f-rdxwm | 0/1   | ContainerCreating | 0        | 8m40s |
| kube-system   | coredns-66bc5c9577-g891x                 | 1/1   | Running           | 0        | 8m41s |
| kube-system   | etcd-minikube                            | 1/1   | Running           | 0        | 8m49s |
| kube-system   | kube-apiserver-minikube                  | 1/1   | Running           | 0        | 8m46s |


```

Image 8 : Démarrage et installation du cluster minikube

7. SCHEMA DESCRIPTIF DE L'INSTALLATION DU CLUSTER MINIKUBE

Voici un schéma descriptif détaillant les éléments nécessaires à connaître avant l'installation du cluster minikube.

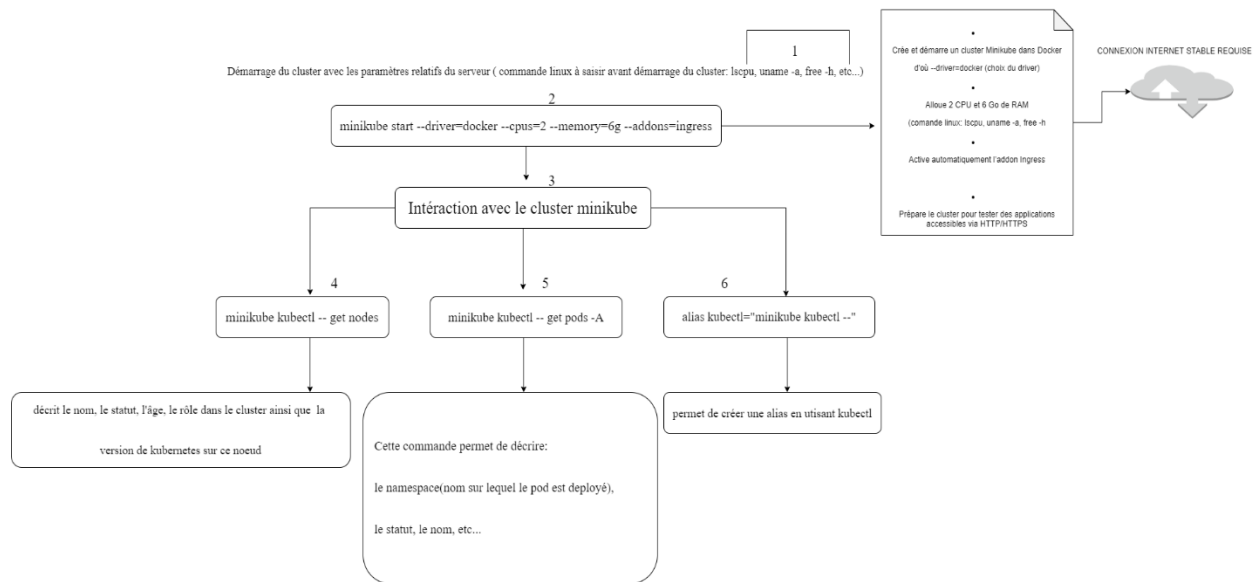


Image 9 : éléments important avant l’installation du cluster minikube.

8. INTERACTION AVEC LE CLUSTER MINIKUBE

Une fois installé, veuillez interagir avec le cluster minikube pour s’assurer du bon fonctionnement de ce dernier.

```
awx@awx-virtual-machine: ~
awx@awx-virtual-machine: $ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
awx@awx-virtual-machine: $ minikube profile list


| PROFILE  | DRIVER | RUNTIME | IP           | VERSION | STATUS | NODES | ACTIVE PROFILE | ACTIVE KUBECONTEXT |
|----------|--------|---------|--------------|---------|--------|-------|----------------|--------------------|
| minikube | docker | docker  | 192.168.49.2 | v1.34.0 | OK     | 1     | *              | *                  |


awx@awx-virtual-machine: $ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready    control-plane   13h   v1.34.0
awx@awx-virtual-machine: $ minikube ip
192.168.49.2
awx@awx-virtual-machine: $
```

Image 10 : interaction avec le cluster minikube après installation

Une fois le cluster minikube installé, veuillez vérifier si vous disposez des éléments ci-après :

1. Un compte github personnel
2. Une paire de clés de ssh au niveau du serveur où sera installé awx-operator

Si les éléments listés ci-dessus ne sont pas présents, veuillez procéder à :

1. La création d'un compte github personnel
2. La création d'une paire de clés ssh moderne.

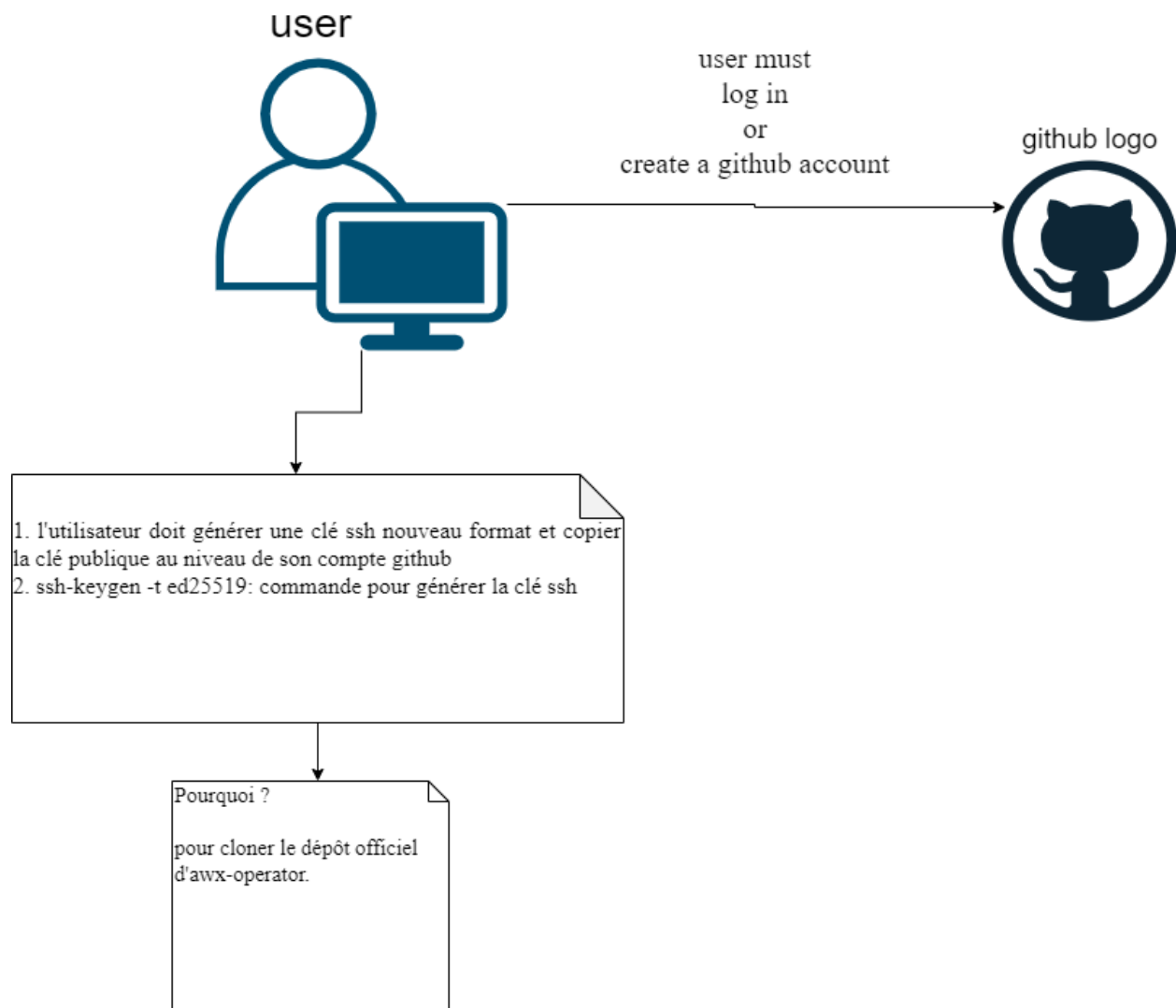


Image 11 : éléments nécessaires avant de cloner le dépôt officielle awx-operator

CETTE DERNIERE PARTIE CONSISTE A L'INSTALLATION DE L'OUTIL AWX-OPERATOR AVEC TOUS LES ELEMENTS NECESSAIRES.

Comme mentionné ci-haut, pour installer awx-operator il faut de préférence avoir :

1. Un compte github personnel
2. Une paire de clés ssh au niveau du serveur
3. Une bonne connectivité internet
4. Assez de ressources (RAM, MEMOIRE, TAILLE DU DISQUE, TYPE DE DISQUE SSD DE PREFERENCE, ETC...)
5. Un cluster kubernetes installé (minikube pour ce tutoriel) et fonctionnel
6. Un système d'exploitation linux de préférence
7. Etc...

9. ETAPES POUR L'INSTALLATION ET LA CONFIGURATION DE L'OUTIL AWX-OPERATOR

Pour installer awx-operator il faut premièrement cloner le dépôt officiel d'awx-operator et choisir le tag (version) de ce dernier:

1. Cloner le dépôt officiel d'awx-operator : git clone [git@github.com:ansible/awx-operator.git](https://github.com:ansible/awx-operator.git)
2. Changer de répertoire en accédant au répertoire awx téléchargé : cd awx-operator
3. Lister les différentes versions d'awx-operator avec la commande : git tag
4. Se déplacer sur la version d'awx-operator choisi avec la commande : git checkout tags/<tag>
5. Exemple : git checkout tags/2.19.0

Pour plus d'info sur les différents tags d'awx-operator veuillez cliquer sur le lien ci-dessous :

<https://github.com/ansible/awx-operator/tags>

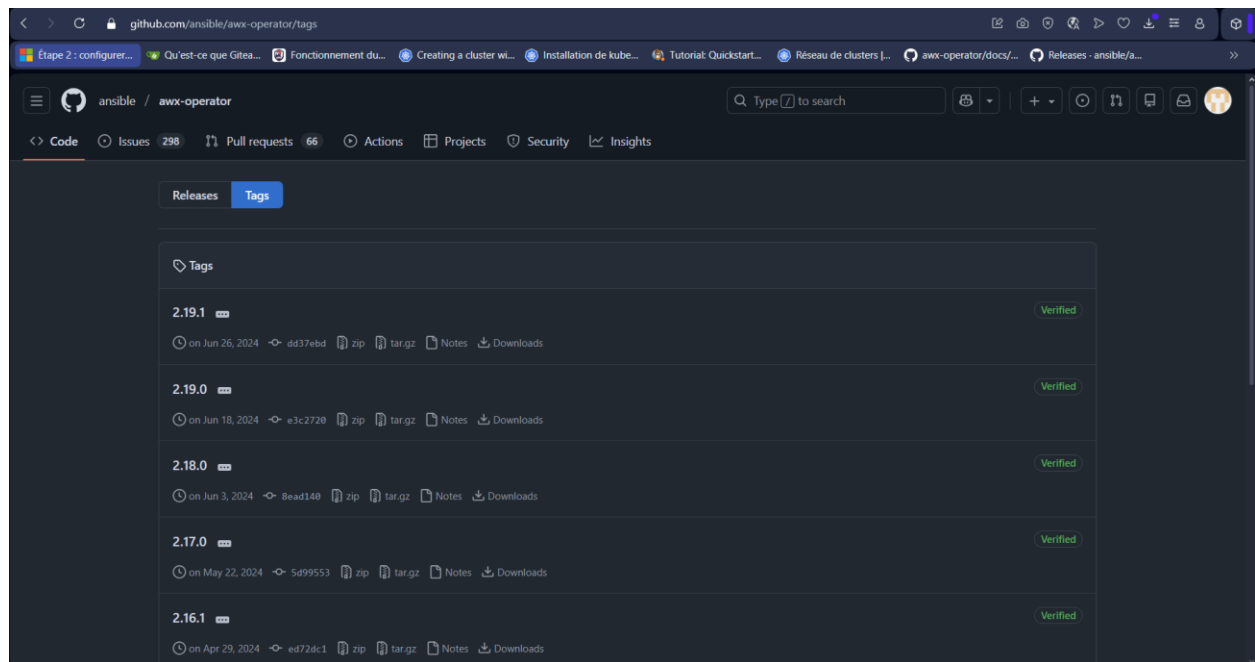


Image 12 : différents tag d'awx-operator

Après avoir choisi le tag, il faut maintenant :

1. Créer le fichier kustomization.yaml manuellement afin de référencer le tag choisi

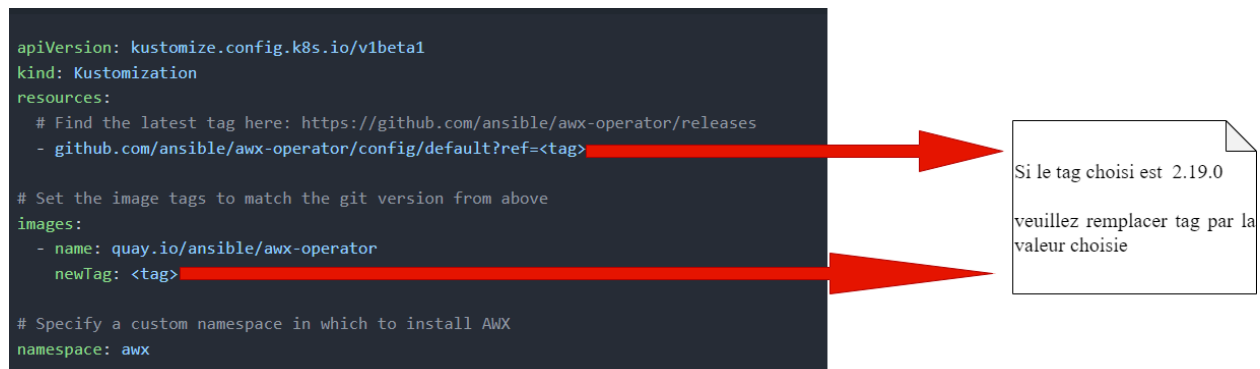
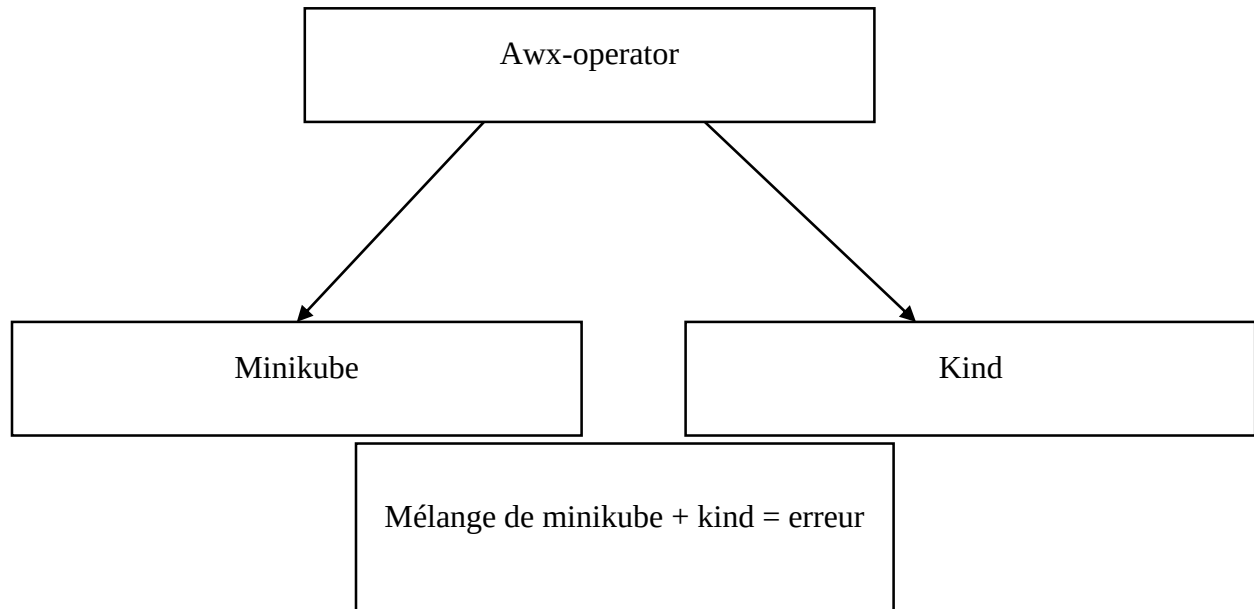


Image 13 : renseignement du tag choisi au niveau du fichier kustomization.yaml

2. Après avoir renseigné le tag (version d'awx-operator) Veuillez appliquer le changement avec la commande : **kubectl apply -k .**

Remarque : l'installation d'awx-operator demande une certaine concentration, éviter de mélanger deux clusters (kind et minikube) cela produit une erreur lors de la saisie d'une commande kubernetes. Il ne faut choisir qu'un seul cluster pour une bonne prise en main et éviter les erreurs.



```
awx@awx-virtual-machine: /awx-operator$ sudo nano kustomization.yaml
awx@awx-virtual-machine: /awx-operator$ kubectl apply -k .
namespace/awx created
Warning: unrecognized format "int32"
customresourcedefinition.apixtensions.k8s.io/awxbackups.awx.ansible.com created
customresourcedefinition.apixtensions.k8s.io/awxmeshingresses.awx.ansible.com created
customresourcedefinition.apixtensions.k8s.io/awxrestores.awx.ansible.com created
serviceaccount/awx-operator-controller-manager created
role.rbac.authorization.k8s.io/awx-operator-awx-manager-role created
role.rbac.authorization.k8s.io/awx-operator-leader-election-role created
clusterrole.rbac.authorization.k8s.io/awx-operator-metrics-reader created
clusterrole.rbac.authorization.k8s.io/awx-operator-proxy-role created
rolebinding.rbac.authorization.k8s.io/awx-operator-awx-manager-rolebinding created
rolebinding.rbac.authorization.k8s.io/awx-operator-leader-election-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/awx-operator-proxy-rolebinding created
configmap/awx-operator-awx-manager-config created
service/awx-operator-controller-manager-metrics-service created
deployment.apps/awx-operator-controller-manager created
awx@awx-virtual-machine: /awx-operator$
```

Image 14 : Application du changement avec la commande **kubectl apply -k .**

Durant l'exécution de cette commande, assurez-vous que votre connexion internet soit très stable.

3. Voir le pod avec la commande : **kubectl get pods -n awx**. Ressayer plusieurs fois cette commande (car un problème de connexion pourrait perturber le chargement)
4. Exécuter la commande **kubectl config set-context --current --namespace=awx** (pour définir le namespace awx comme namespace par défaut pour le contexte Kubernetes actuellement actif.)
5. Vérifier si le fichier awx-demo.yml existe. S'il existe ok. Sinon veuillez le créer

```
---
apiVersion: awx.ansible.com/v1beta1
kind: AWX
metadata:
  name: awx-demo
spec:
  service_type: nodeport
```

Image 15 : création du fichier awx-demo.yml si ce dernier n'existe pas

6. Veuillez modifier le fichier kustomization.yaml afin d'ajouter le fichier awx-demo.yml au niveau de la ressource

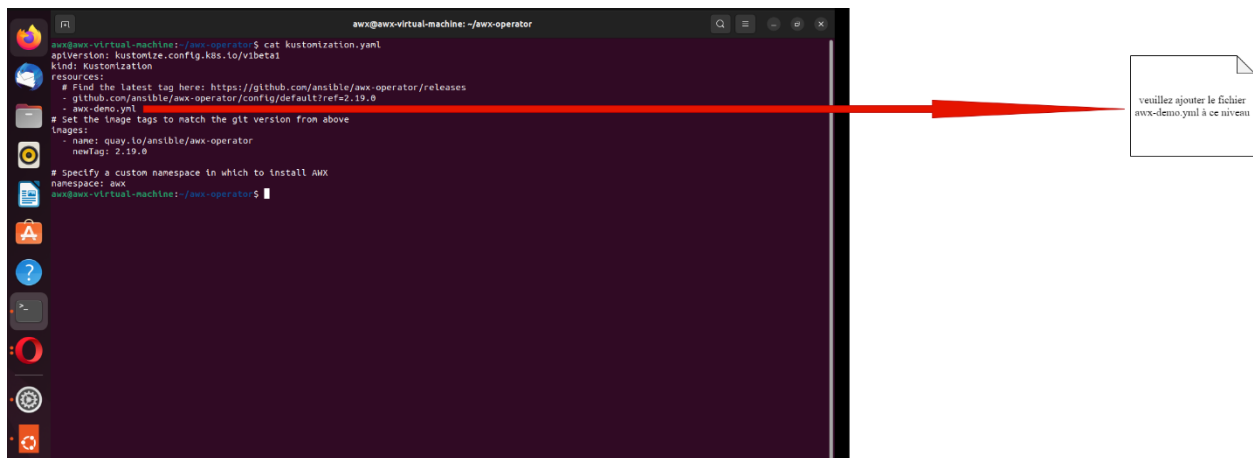


Image 16 : rajout du fichier awx-demo.yml au niveau de la ressource

Après l'ajout du fichier awx-demo.yml, veuillez saisir la commande **kubectl apply -k .** pour appliquer le changement.

7. Commande à exécuter

A la fin de cette commande, awx-operator sera déjà disponible. Veuillez saisir la commande ci-après pour voir les différents changements.

Nota : essayer plusieurs fois (assurez-vous d'avoir une connexion internet stable) pour le chargement d'images et autres éléments (tels que la création de différents pods etc...)

Les deux commandes sont valides

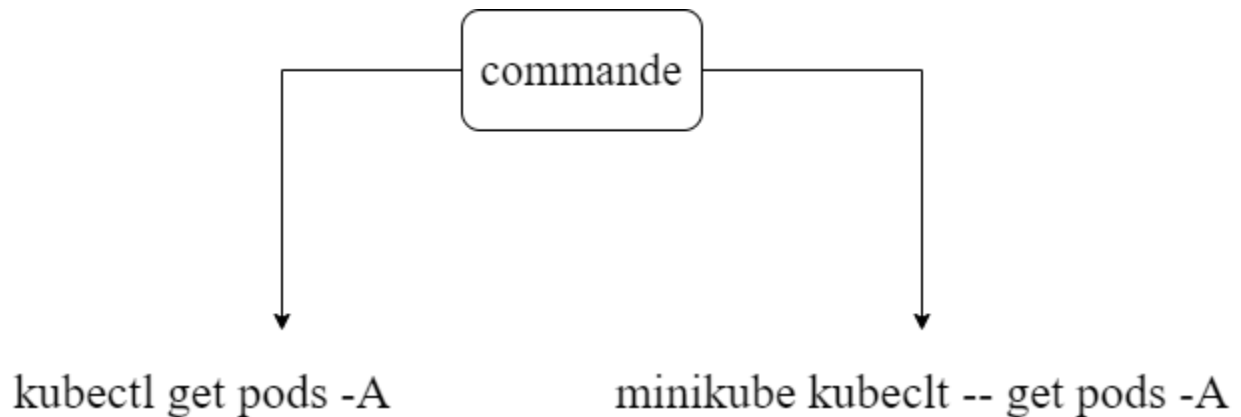


Image 17 : commande à saisir pour voir les différents états de pods, nom de namespaces etc...

```
minikube status
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

awx@awx-virtual-machine: $ kubectl get pods -A
NAME                                READY   STATUS    RESTARTS   AGE
minikube                            0/1     Completed 0           13h
awx                                 1/1     Running   0           5h9m
awx                                 4/4     Running   0           5h7m
awx                                 3/3     Running   0           5h7m
awx-operator-controller-manager-6c9f5c9d5d-qcmv 2/2     Running   2 (5h18m ago) 5h43m
ingress-nginx-admission-create-sr56h           0/1     Completed 0           13h
ingress-nginx-admission-patch-fz964            0/1     Completed 0           13h
ingress-nginx                               1/1     Running   1 (5h18m ago) 13h
ingress-nginx-controller-9cc49f96f-rdwm        1/1     Running   2 (5h18m ago) 13h
kube-system                                1/1     Running   2 (5h18m ago) 13h
kube-system                                1/1     Running   2 (5h18m ago) 13h
kube-system                                1/1     Running   2 (5h18m ago) 13h
kube-system                                1/1     Running   2 (5h18m ago) 13h
kube-system                                1/1     Running   2 (5h18m ago) 13h
kube-system                                1/1     Running   5 (5h17m ago) 13h
awx@awx-virtual-machine: $ minikube profile list
PROFILE  DRIVER  RUNTIME  IP          VERSION  STATUS  NODES  ACTIVE PROFILE  ACTIVE KUBECONTEXT
minikube docker  docker  192.168.49.2 v1.34.0  OK      1      *               *
```


Image 18 : exécution de différentes commandes pour voir l'état des éléments tels que (pods, namespaces, node, statut du cluster, liste de profile, etc...)

8. Affichage de l'interface d'accueil

Pour afficher les informations relatives à AWX-operator (adresse IP, numéro port), veuillez saisir la commande : **minikube service list**

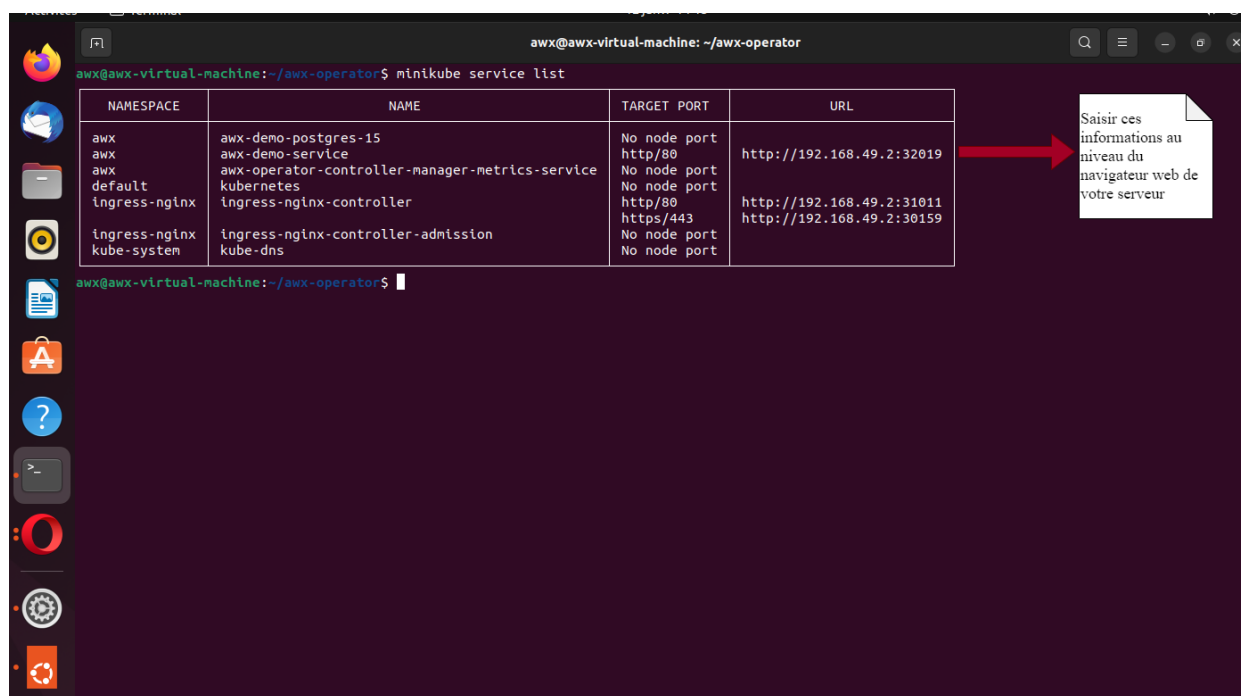


Image 19 : Récupération d'adresse IP + numéro port pour lancer awx-operator

Après cela, l'interface d'awx-operator sera opérationnelle au niveau du navigateur.

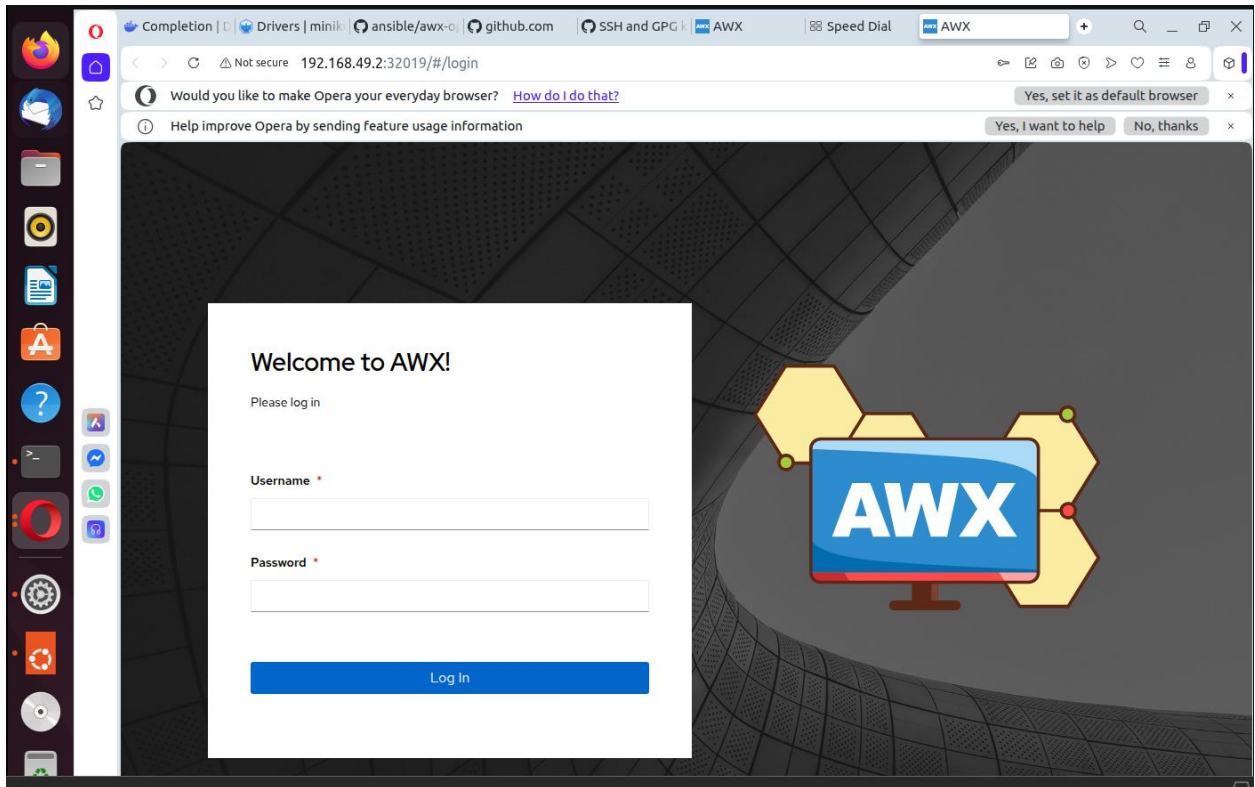


Image 20 : interface d'accueil d'awx-operator

Pour récupérer le mot de passe afin de se connecter au dashboard principal veuillez procéder de la manière ci-après :

1. **Kubectrl get secrets -n awx | minikube kubectl -- get secrets -n awx**
2. **Kubectrl get namespaces | minikube kubectl -- get namespaces -n awx**
3. **kubectrl get secret <nom de la resource>-admin-password -o jsonpath="{.data.password}" | base64 --decode ; echo | minikube kubectl -- get secret <nom de la resource>-admin-password -o jsonpath="{.data.password}" | base64 --decode ; echo**

Nota: les deux commandes fonctionnent. Veuillez choisir celle qui vous convient.

Adopter les choix du secret en fonction de votre sortie.

Le nom d'utilisateur par défaut est admin

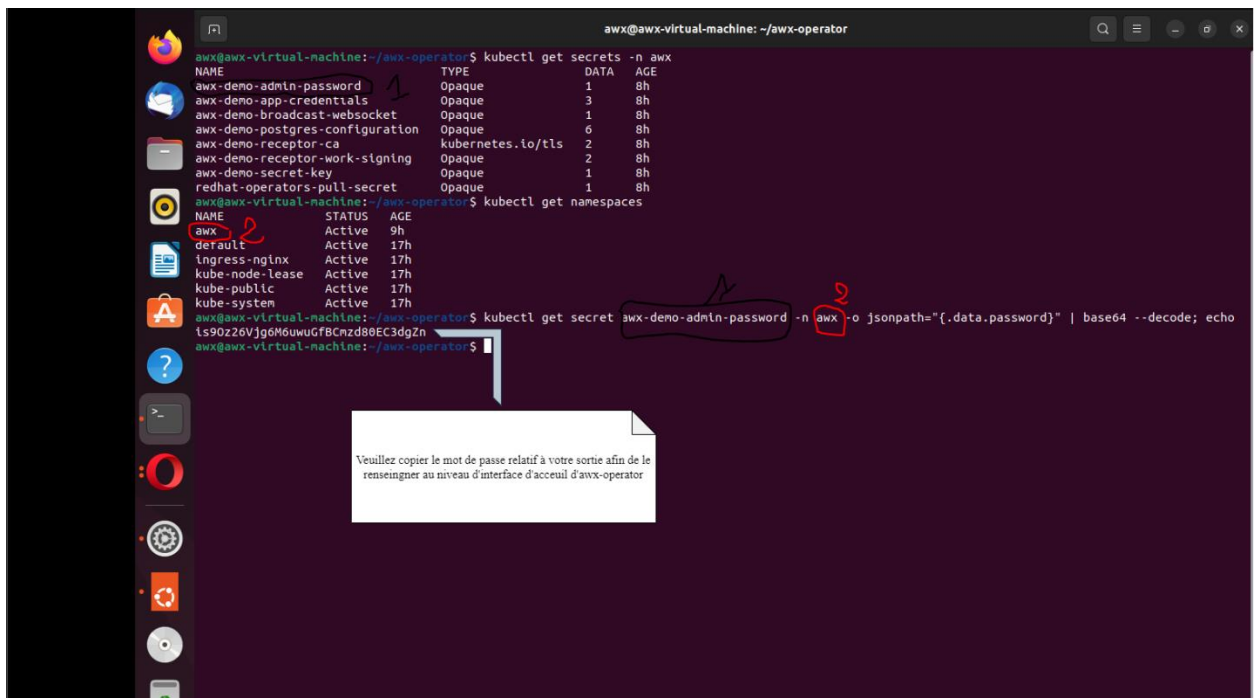


Image 21 : récupération d'éléments nécessaires pour se connecter à AWX-operator

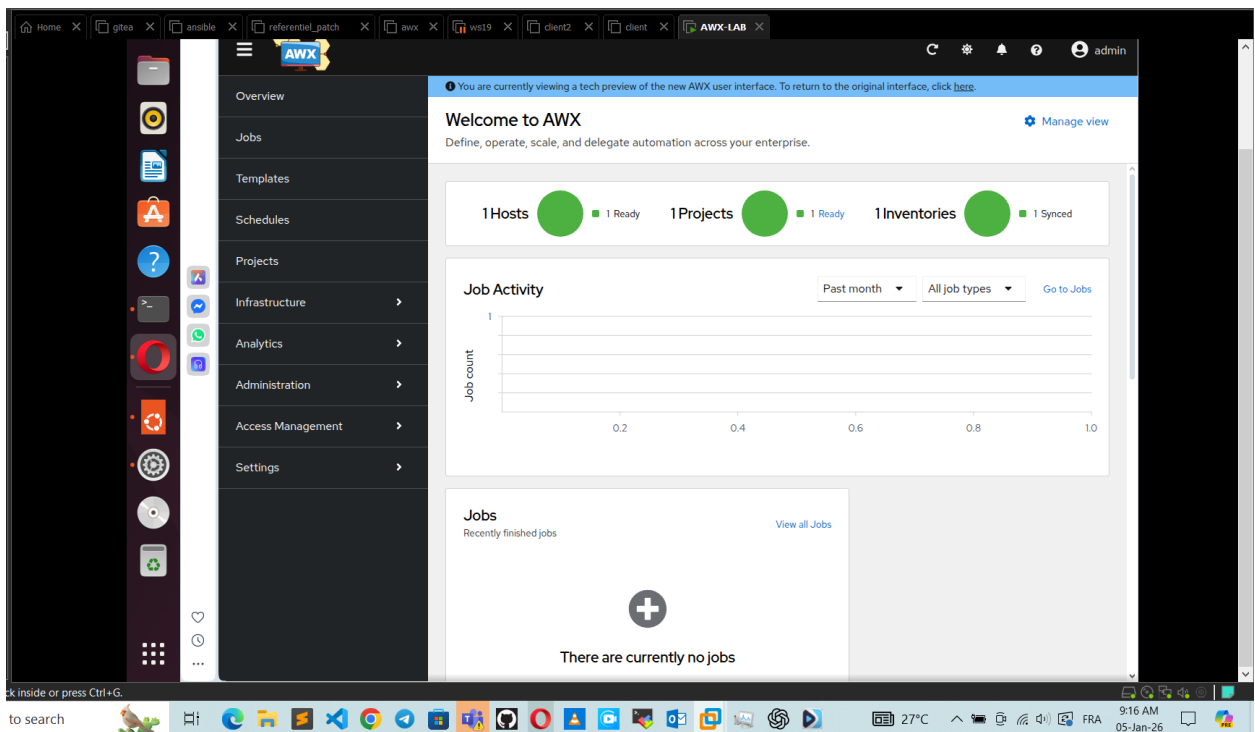


Image 22: Dashboard principal d'awx-operator

QUELQUES CAS COURANTS PENDANT L'INSTALLATION D'AWX-OPERATOR

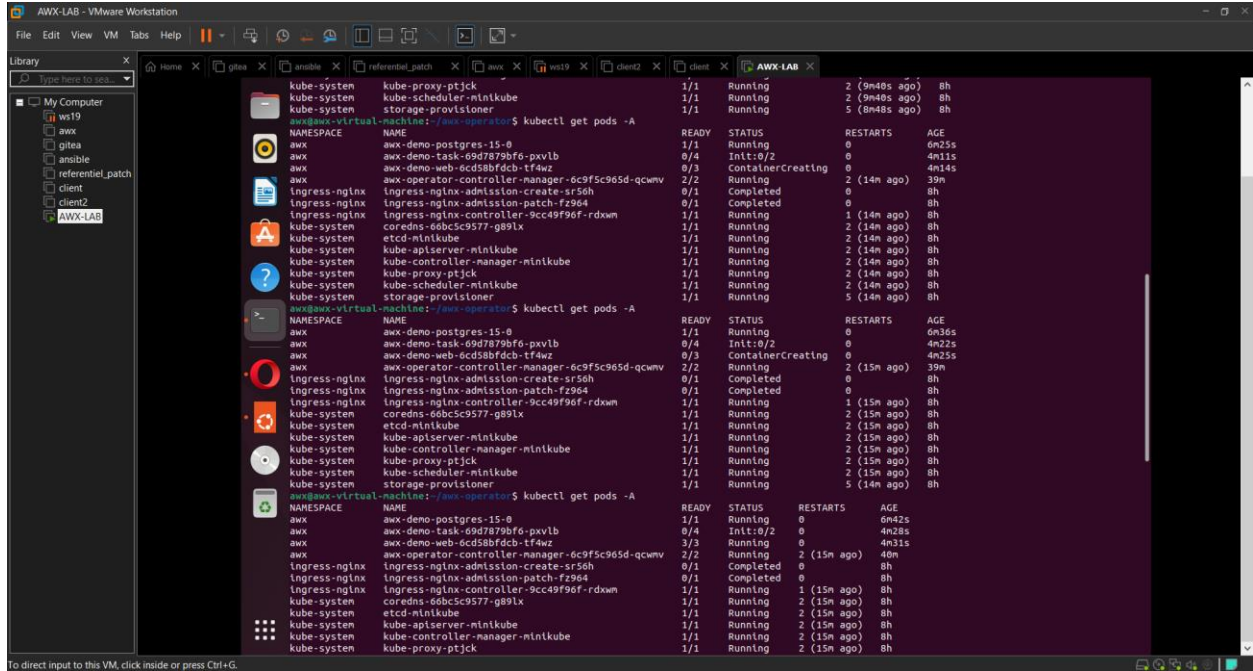


Image 23

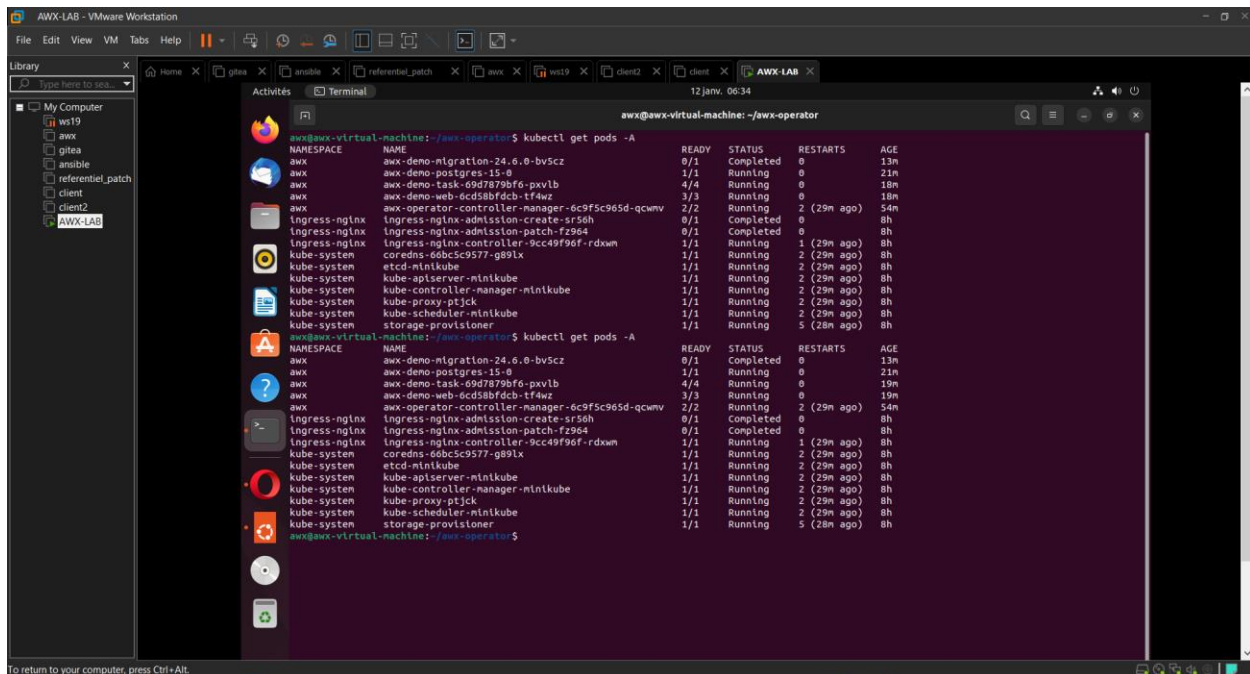


Image 24

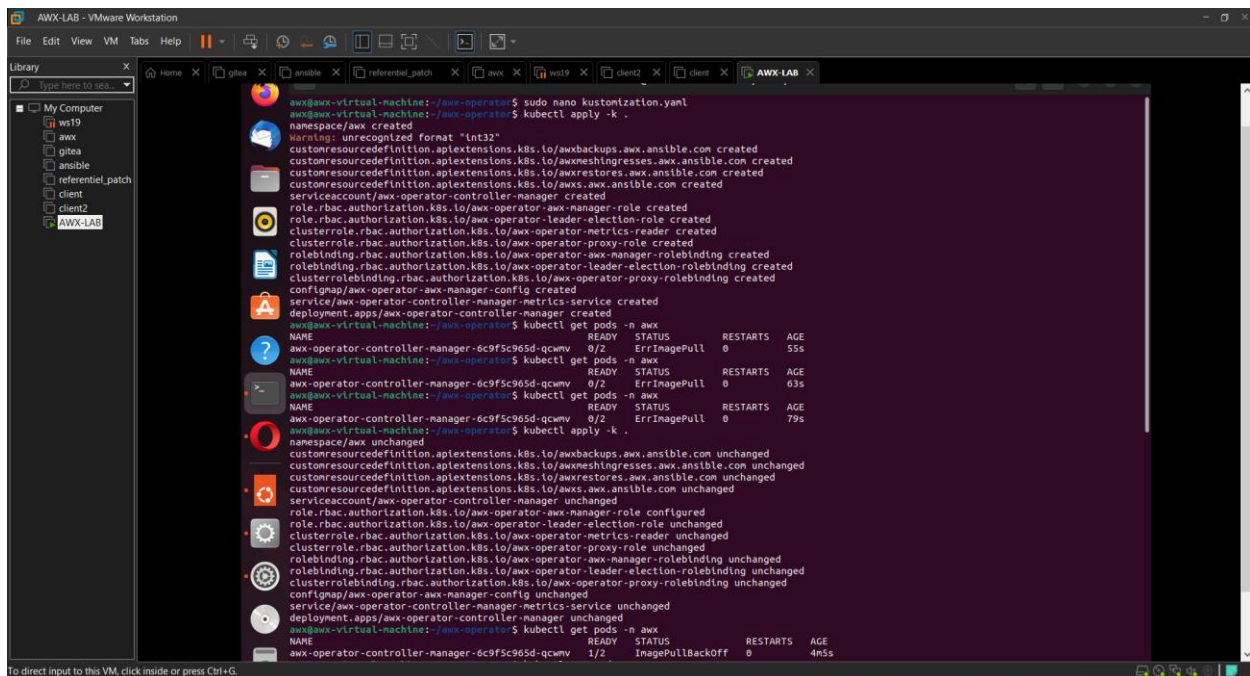


Image 25

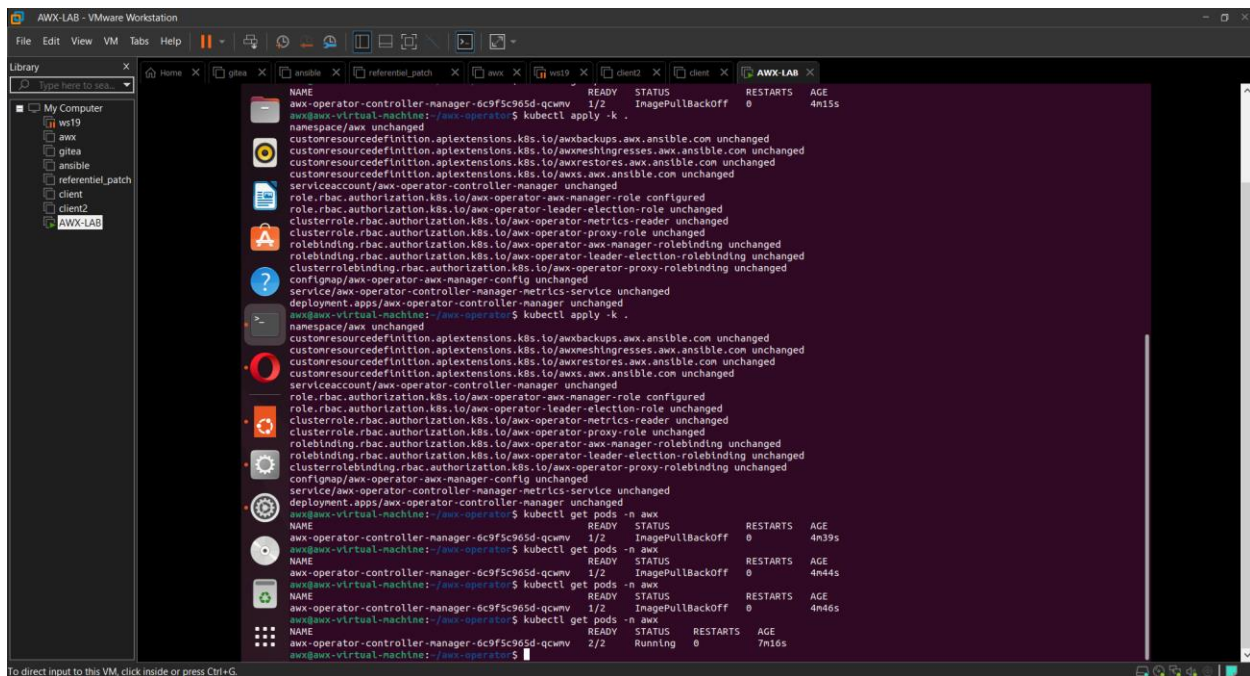


Image 26

Les différentes captures démontrent à quel point une connexion internet stable est requise pour appliquer les différents changements.

En cas d'oubli du mot de passe admin veuillez procéder de la manière ci-après pour changer le mot de passe facilement :

1. Vérifier le pods avec la commande ci-après : **minikube kubectl -- get pods -n awx**
2. Veuillez accéder au pod pour manager le changement du mot de passe sans problème.

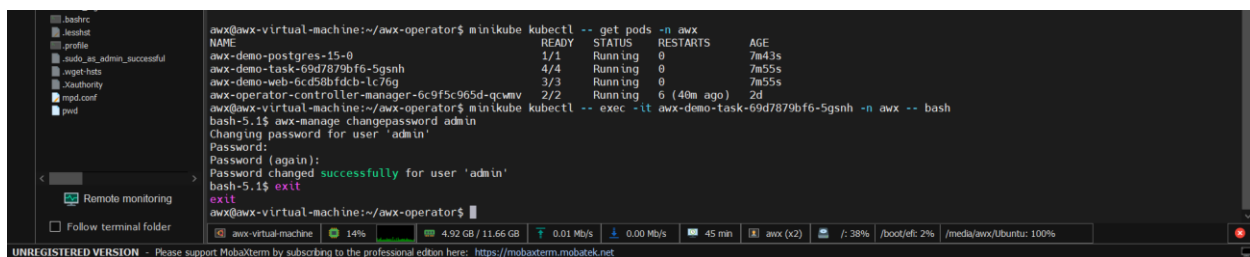


Image 27 : Réinitialisation du mot de passe oublié

CONCLUSION

Nous voici arriver au terme de ce tutoriel, tout au long de ce tutoriel nous avons vu l'importance du dimensionnement d'un serveur linux, qui constitue un élément important permettant le bon fonctionnement de ce dernier afin d'installer, d'utiliser les différents outils sans problème. Et nous avons installé un cluster minikube afin de déployer l'outil open source AWX-OPERATOR. Outil qui permet de manager une infrastructure informatique de manière plus structurée, plus agile tout en permettant l'intégration de différentes technologies.

Contents

DEPLOIEMENT D'AWX-OPERATOR AVEC CLUSTER MINIKUBE	1
INTRODUCTION.....	1
1. DIMENSIONNEMENT DU SERVEUR.....	1
2. COMMENT DECOUVRIR LES INFORMATIONS RELATIVES A SON SERVEUR ?.....	1
3. DESCRIPTION DE L'OUTIL AWX-OPERATOR.....	2
4. MODE DE FONCTIONNEMENT ET CHOIX DU CLUSTER	2
5. INSTALLATION DU CLUSTER MINIKUBE	3
5.1 Dimensionnement du serveur	3
5.2 Vérification et Installation de docker.....	5
5.3 Installation de docker engine.....	5
6. INSTALLATION DE MINIKUBE	7
7. SCHEMA DESCRIPTIF DE L'INSTALLATION DU CLUSTER MINIKUBE	9
8. INTERACTION AVEC LE CLUSTER MINIKUBE	10
CETTE DERNIERE PARTIE CONSISTE A L'INSTALLATION DE L'OUTIL AWX- OPERATOR AVEC TOUS LES ELEMENTS NECESSAIRES.	12
9. ETAPES POUR L'INSTALLATION ET LA CONFIGURATION DE L'OUTIL AWX- OPERATOR.....	12
1. Créer le fichier kustomization.yaml manuellement afin de référencer le tag choisi.....	13
2. Après avoir renseigné le tag (version d'awx-operator) Veuillez appliquer le changement avec la commande : kubectl apply -k	13
3. Voir le pod avec la commande : kubectl get pods -n awx . Ressayer plusieurs fois cette commande (car un problème de connexion pourrait perturber le chargement).....	15
4. Exécuter la commande kubectl config set-context --current --namespace=awx (pour définir le namespace awx comme namespace par défaut pour le contexte Kubernetes actuellement actif.).....	15
5. Vérifier si le fichier awx-demo.yml existe. S'il existe ok. Sinon veuillez le créer.....	15
6. Veuillez modifier le fichier kustomization.yaml afin d'ajouter le fichier awx-demo.yml au niveau de la ressource.....	15
7. Commande à exécuter.....	16
8. Affichage de l'interface d'accueil.....	17
QUELQUES CAS COURANTS PENDANT L'INSTALLATION D'AWX-OPERATOR.....	20
En cas d'oubli du mot de passe admin veuillez procéder de la manière ci-après pour changer le mot de passe facilement :	22

CONCLUSION	23
-------------------------	-----------